



Activity File: SQL Refresher and Unintended SQL Queries

In this activity, you will work as an application security engineer for the bioengineering company, Replicants.

- Replicants recently received an anonymous email stating that several web vulnerabilities have been identified on the company website. Unfortunately, the anonymous email didn't indicate what they were.
- You are first tasked with determining whether the Replicants website is vulnerable to SQL injection. This is one of the most dangerous vulnerabilities and could expose confidential information.
- You will test a page on the Replicants website that connects to its database, where an employee can confirm their first and last name by looking up their user id.
- Before testing for SQL injection directly on the webpage, you will need to design several SQL queries to test directly against the database, which is represented by a website called DB Fiddle.
- The queries you design will be used in the next activity to create payloads to test for SQL injection vulnerabilities on the website.

Setup

Access DB Fiddle using the following link: [DB Fiddle: SQL Injection Practice](#).

- For the purposes of this lesson, the webpage will represent a mock version of the Replicants database where you can create and test several SQL queries.
- On the left side of this page is the **Schema SQL** panel, which builds the mock database for Replicants. Do not modify any of the data in this section.
- On the right side of the page is the **Query SQL** panel, which contains the steps to complete the activity, as well as where the queries are modified:

```
Run Update Fork Load Example Star PRO Embed PRO Collaborate Sign in Have any feedback?
```

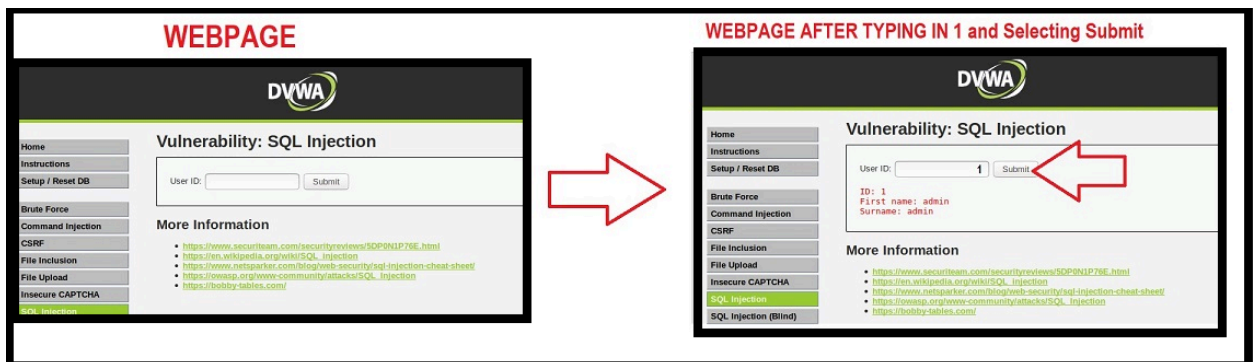
Schema SQL

```
1 # The following queries below designs a simulated database and table
2   for CyberDefenders. Do not modify or change any of the data below.
3 CREATE TABLE IF NOT EXISTS `users` (
4   `user_id` int(6) unsigned NOT NULL,
5   `first_name` varchar(200) NOT NULL,
6   `last_name` varchar(200) NOT NULL,
7   `password` varchar(200) NOT NULL
8 );
9
10 INSERT INTO `users` (`user_id`, `first_name`,
11   `last_name`, `password`) VALUES
12   ('1', 'admin', 'admin', '5f4dcc3b5aa765d61d8327deb882cf99'),
13   ('2', 'gordon', 'brown', 'e99a18c428cb38d5f260853678922e03'),
14   ('3', 'hack', 'me', '8d3533d75ae2c3966d7e0d4fcc69216b'),
15   ('4', 'pablo', 'picasso', '0d107d09f5bbe40cade3de5c71e9e9b7'),
16   ('5', 'bob', 'smith', '5f4dcc3b5aa765d61d8327deb882cf99');
```

Query SQL

```
1 #####
2 ##### !!! Start Here !!! #####
3 #####
4
5 # BACKGROUND:
6
7 -- In this activity, you work as an Application Security Engineer
8   with Cyber Defenders.
9
10 -- Your company recently received an anonymous email stating that
11    several web vulnerabilities have been identified on your
12    company's website. Unfortunately, the anonymous email didn't
13    indicate what they are.
14
15 -- As there are many potential web vulnerabilities, you are first
16    tasked with determining if your company's website is vulnerable
17    to SQL injection as this is one of the most dangerous
18    vulnerabilities which could expose Cyber Defenders confidential
19    information.
```

- For additional context, the following image illustrates how the Replicants webpage pulls data from the database:



- Note that you will access and interact with this webpage directly in the next activity.

Instructions

- Locate the **Start Here** text on the top right of the DB Fiddle webpage.
- Follow the directions on DB Fiddle to create five SQL queries. Note that the fourth and fifth steps are bonuses.
 - For reference, here is another [copy of the steps](#).
- Create and run each SQL query as directed.

Don't forget to keep these SQL queries. You will be using them in your next activity!